SmartWatchVM — Language & Virtual Machine for Smartwatches

Motivation, Features, Architecture and Practical Examples

SmartWatchVM and its VM were created by the Guilherme Paraiso.

Motivation

Current Context

Cardiovascular sports boom: activities such as running, cycling, swimming and Ironman have grown significantly in popularity, creating a larger audience of athletes who rely on wearable data for training and performance monitoring.

Wearable market growth: the smartwatch and fitness-tracking ecosystem has been expanding rapidly, with an estimated growth rate above 25% per year between 2020 and 2025, increasing the relevance of tools built for these devices.

Need for personalization: as athletes become more data-driven, they require personalized algorithms capable of addressing their unique training routines, performance goals and monitoring needs.

Identified Problem

Despite this expanding market and the increasing specialization of athletes, current smartwatches still offer only generic, predefined functionalities and do not provide programming capabilities that would allow tailored, athlete-specific training logic.

Proposed Solution

Create a specialized language and virtual machine designed to:

• Enable the programming of custom training logic aligned with individual athletic needs.

• Provide direct access to real-time sensor data available in modern smartwatches.

• Support the implementation of sport-specific algorithms that enhance monitoring and decision-making during training.

Language Characteristics (Simplified Version)

1. Athlete-Friendly Syntax

The language was designed to be easy to understand even for users with no programming experience. Its structure allows clear, intuitive rules to be written in a natural way, such as:

Example:

```
WHEN HEARTRATE > 180 THEN
    NOTIFY "Risk zone!"
    REDUCE_INTENSITY
ENDWHEN
```

2. Integrated Sensors

The language provides direct access to the same sensors commonly used by real smartwatches during training, including:

• Heart rate
• Step count
• Battery level
• Current time

Programs can read these sensor values at any moment to make real-time, intelligent decisions during physical activities.

3. Control Structures Designed for Sports

The control structures were created with training routines in mind, especially repetitive or interval-based workouts. This enables the creation of fully automated training sequences.

Example:

```
LOOP interval_count < 10 DO
    HIGH_INTENSITY_PHASE
    RECOVERY_PHASE
    interval_count = interval_count + 1
ENDLOOP
```

4. Fully Powerful (Turing-Complete)

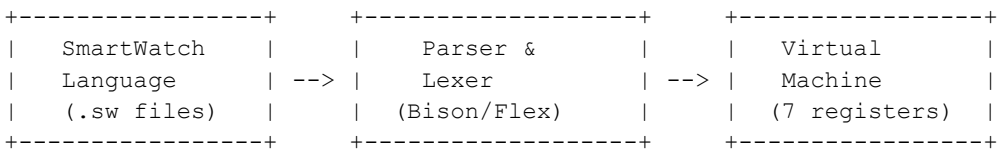
Even with its simple and friendly syntax, the language is powerful enough to express any type of sports-related logic, such as:

• Advanced calculations
• Performance-oriented algorithms
• Continuous real-time sensor processing

This means users can program virtually any behavior they want their smartwatch to perform.

Technical Architecture

Virtual Machine Structure



This diagram represents the full flow of the SmartWatchVM execution pipeline. Programs written in .sw files are processed by a lexer and parser built with Flex and Bison, and then executed by the virtual machine, which operates using a minimal but effective set of seven registers.

Specialized Registers

Register	Type	Sports Function
TIME	Read/Write	Controls training time and clock values
ALARM	Read/Write	Handles custom alerts
HEART	Read Only	Real-time heart rate monitoring
STEP	Read Only	Step counting during activity
TRACK	Read/Write	Music playback control
BT	Read/Write	Bluetooth connection management
TIMER	Read/Write	Training and interval timing

Practical Examples

1. Heart Rate Zone Monitor

```
POWERON

# Training zone definitions
fat_burn_zone = 120
aerobic_zone = 150
anaerobic_zone = 170

WHEN HEARTRATE < fat_burn_zone THEN
    NOTIFY "Increase intensity"
    SHOW "Zone: Recovery"
ENDWHEN

WHEN HEARTRATE >= fat_burn_zone AND HEARTRATE < aerobic_zone THEN
    SHOW "Zone: Fat Burning"
ENDWHEN

WHEN HEARTRATE >= aerobic_zone AND HEARTRATE < anaerobic_zone THEN
    SHOW "Zone: Aerobic"
ENDWHEN

WHEN HEARTRATE >= anaerobic_zone THEN
    NOTIFY "Risk Zone - Reduce intensity!"
    SHOW "Zone: Anaerobic"
ENDWHEN
```

```
POWEROFF
```

Simplified VO_2 Max Formula

```
vo2_estimated = (max_hr * 0.8) - (age * 0.2) + (weight * 0.1)

SHOW "Estimated VO2 Max:"
SHOW vo2_estimated

WHEN vo2_estimated > 50 THEN
    NOTIFY "Excellent conditioning!"
ELSE
    NOTIFY "Keep training!"
ENDWHEN

POWEROFF
```

Advanced Mathematical Examples

Fibonacci for Training Progression

```
# Fibonacci sequence for training load progression
n = 10
a = 0
b = 1
week = 1

SHOW "Fibonacci Progression:"

LOOP week <= n DO
    SHOW a
    NOTIFY "Training week"

    increment = a
    SHOW increment

    temp = a + b
    a = b
    b = temp
    week = week + 1
ENDLOOP
```

Technical Details

1. Minsky Machine Inspiration

- Theoretical basis for Turing-completeness
- Minimum 2 registers for universal computation
- Efficient implementation on resource-limited devices

2. Compilation Process

```
# Development with industrial tools
bison -d parser.y     # Syntax analyzer
flex lexer.l          # Lexical analyzer
gcc -o vm *.c -lfl    # Final compilation
```

3. Verification System

- Static analysis of undefined variables
- Detection of unused labels
- Compile-time type checking

4. Wearable Optimizations

- Low power consumption
- Efficient processing on limited hardware
- Fast response time for real-time applications

Real-World Applications

For Runners

- Pace alert programming

- Pace per km calculator
- Hydration alerts

For Cyclists

- Power management
- Cadence alerts
- FTP (Functional Threshold Power) calculator

For Swimmers

- Lap counting
- Stroke efficiency analysis
- Custom rest intervals

For Triathletes

- Transitions between sports
- Energy management between stages
- Nutrition and hydration alerts

Conclusion

Potential Impact

- Democratization: amateur athletes can program their own algorithms
- Personalization: specific solutions for each sport and athlete
- Innovation: new category of “programmable sports software”

Resources

- Repository**:** https://github.com/guiparaiso/SmartWatchVM